

# Spec & Architecture Document

## AI-Powered Web Application for Philippine Disaster Preparedness and Evacuation Guide

**Version:** 1.0 (Draft) · **Status:** For Engineering Review · **Derived from:** PRD v1.0

### 1. Context & Goal

This application helps people in disaster-prone areas of the Philippines know exactly what to do and where to go before, during, and after a typhoon, flood, or landslide. It takes official but hard-to-parse hazard bulletins (from PAGASA/PHIVOLCS) and turns them into short, plain-language instructions in the user's own language (Tagalog, Bisaya, or English), tied to their specific barangay or city. It also shows nearby evacuation centers and whether they're currently open, full, or closed, using a mix of local government updates and reports from other users. The two main kinds of people who use it are ordinary residents trying to keep their own family safe, and "community relay" figures — teachers, health workers, barangay staff — who pass information on to their neighbors, often by forwarding a link or a screenshot in a Facebook or Messenger group. The core problem this solves is the gap between "a typhoon is coming" and "I know what to actually do about it, right now, where I live" — a gap that today is filled by guesswork, rumor, or information that arrives too late or too generic to act on.

### 2. Inputs & Outputs

#### 2.1 Core Domain Types (TypeScript)

```
// — Location —————
interface LocationRef {
  barangayCode: string; // PSGC (Philippine Standard Geographic Code), e.g.
  "137404001"
  barangayName: string;
  cityMunicipality: string;
  province: string;
  region: string;
  lat?: number; // optional, only if user grants precise location
  lng?: number;
}

// — Language —————
type LanguageCode = "tl" | "ceb" | "en"; // Tagalog, Bisaya/Cebuano, English

// — Hazard Bulletin (ingested from PAGASA/PHIVOLCS) —————
interface HazardBulletin {
  id: string;
  source: "PAGASA" | "PHIVOLCS" | "OCD" | "MANUAL_ADMIN";
  hazardType: "TYPHOON" | "FLOOD" | "STORM_SURGE" | "LANDSLIDE" | "EARTHQUAKE";
  issuedAt: string; // ISO 8601 UTC
  validUntil: string; // ISO 8601 UTC
  affectedAreas: LocationRef[]; // may be province/region-level, not just barangay
```

```

    severity: 1 | 2 | 3 | 4 | 5; // normalized signal/alert level
    rawText: string;           // original bulletin text, unmodified, kept for audit
}

// — AI-Generated Guidance —————
interface GuidanceRequest {
    location: LocationRef;
    language: LanguageCode;
    requestedAt: string;
}

interface GuidanceResponse {
    bulletinId: string;           // which bulletin this guidance is derived from
    generatedAt: string;
    language: LanguageCode;
    phase: "NOW" | "NEXT_24H" | "DURING_IMPACT" | "AFTERMATH";
    summary: string;              // 1-2 sentence plain-language headline
    actionItems: string[];        // short, imperative bullet points
    sourceAttribution: string;    // e.g. "Based on PAGASA bulletin issued 2026-07-01 14:00"
    isFallback: boolean;          // true if AI generation failed and static content was
served
}

// — Evacuation Centers —————
type EvacStatus = "OPEN" | "FULL" | "CLOSED" | "UNKNOWN";

interface EvacuationCenter {
    id: string;
    name: string;
    location: LocationRef;
    lat: number;
    lng: number;
    capacity?: number;
    status: EvacStatus;
    statusUpdatedAt: string;
    statusSource: "LGU_ADMIN" | "COMMUNITY_REPORT" | "DEFAULT_UNKNOWN";
    reportCount: number;          // number of corroborating community reports in last 6h
}

// — Community Reports —————
interface CommunityReport {
    id: string;
    type: "EVAC_STATUS" | "ROAD_CONDITION" | "OTHER_HAZARD";
    targetEvacCenterId?: string;  // required if type === "EVAC_STATUS"
    location: LocationRef;
    message: string;              // max 280 chars, free text
    reportedStatus?: EvacStatus;  // required if type === "EVAC_STATUS"
    submittedAt: string;
    clientHash: string;           // anonymized device/session fingerprint for spam
throttling
}

```

```
// — Shareable Alert —————
interface ShareableAlert {
  guidanceResponseId: string;
  shareText: string;           // pre-formatted, ready to paste into Messenger/Facebook
  shareUrl: string;           // deep link back to this guidance in the app
}
```

## 2.2 Key API Endpoints (request/response shape)

| Endpoint                     | Method            | Input                                         | Output                            |
|------------------------------|-------------------|-----------------------------------------------|-----------------------------------|
| /api/guidance                | POST              | GuidanceRequest                               | GuidanceResponse                  |
| /api/evac-centers            | GET               | { lat, lng, radiusKm } OR {<br>barangayCode } | EvacuationCenter[]                |
| /api/evac-centers/:id/status | PATCH (LGU admin) | { status: EvacStatus,<br>updatedBy: string }  | EvacuationCenter                  |
| /api/reports                 | POST              | CommunityReport (minus id)                    | { id: string, accepted: boolean } |
| /api/bulletins/latest        | GET               | { locationRef }                               | HazardBulletin[]                  |
| /api/share                   | POST              | { guidanceResponseId }                        | ShareableAlert                    |

## 2.3 Offline Bundle Format

```
interface OfflineBundle {
  bundleVersion: string;
  generatedAt: string;
  validForRegion: string;           // province or region code, coarse-grained
  staticChecklists: Record<LanguageCode, string[]>; // generic prep checklists, hazard-agnostic
  hazardSpecificGuidance: Record<
    "TYPHOON" | "FLOOD" | "STORM_SURGE" | "LANDSLIDE",
    Record<LanguageCode, string[]>
  >;
  lastKnownEvacCenters: EvacuationCenter[]; // snapshot at bundle generation time, may be stale
}
```

# 3. Behavior Rules

## 3.1 Guidance Generation

- **No bulletin available for the user's location:** Do not fabricate hazard-specific guidance. Return `GuidanceResponse` with `isFallback: true`, generic hazard-preparedness content, and a clear message that no active bulletin applies to this location right now.
- **Bulletin exists but AI generation fails or times out (>5s):** Serve a pre-written static fallback for that `hazardType` + `severity` combination from the offline bundle content. Always set `isFallback:`

true so the UI can visually distinguish AI-personalized from static fallback content.

- **Bulletin exists but is stale ( validUntil has passed):** Do not present it as current. Show the most recent applicable bulletin only if still within validUntil ; otherwise show "no active advisory" state, not an expired one silently reused.
- **Conflicting or overlapping bulletins for the same location (e.g., both a flood and typhoon bulletin active):** Surface both, ranked by severity descending, not merged into one AI-generated response — never let the AI silently pick one and discard the other.
- **AI output validation:** Every generated actionItems list must be checked against a banned-content filter (no medical dosing advice, no unverifiable claims, no content unrelated to the bulletin) before being served. Reject and fall back to static content if validation fails.
- **Location not resolvable** (user types a barangay name that doesn't match any PSGC record): Prompt for disambiguation with fuzzy-matched suggestions; never silently default to a random or "nearest guess" location for hazard guidance.

### 3.2 Evacuation Center Data

- **No centers found within default radius:** Expand search radius incrementally (5km → 15km → 30km) and label results with actual distance; never return an empty state without at least attempting a wider search.
- **Status conflict between LGU admin input and community reports:** LGU admin ( statusSource: "LGU\_ADMIN" ) always takes precedence over community reports for status , regardless of recency, but the UI must show community reports as supplementary context (e.g., "LGU says open, but 4 recent reports say full").
- **Single unverified community report changing status:** Do not let a single report flip a center's displayed status. Require either (a) an LGU admin update, or (b) ≥3 corroborating community reports within a 6-hour window with the same reportedStatus , before updating the displayed status .
- **Stale status** ( statusUpdatedAt older than 24 hours during an active bulletin): Display status with a visible "last confirmed X hours ago — may be outdated" warning rather than presenting it as current.
- **No status data at all:** Default to UNKNOWN , never default to OPEN . Absence of data must never imply safety.

### 3.3 Community Reporting

- **Rate limiting / spam prevention:** Throttle by clientHash — max 10 reports per hour per client. Reject and return a clear error, don't silently drop.
- **Profanity / abuse / clearly false content:** Basic keyword/heuristic filtering rejects obviously abusive submissions at intake; flagged-but-ambiguous content is accepted but marked needsReview for later moderation, not blocked outright (avoid over-censoring genuine reports during a crisis).
- **Malformed report** (missing required fields for its type ): Reject with a specific validation error, not a generic failure.

### 3.4 Offline Behavior

- **Connectivity lost mid-session:** App must detect offline state and switch UI to serve from OfflineBundle automatically, with a visible "offline mode — showing last saved information from [timestamp]" banner.
- **Offline bundle never downloaded** (e.g., first-time user with no connectivity): Show an explicit empty/error state instructing the user to connect briefly to download offline content, rather than a blank screen.
- **Bundle staleness:** If OfflineBundle.generatedAt is more than 7 days old, prompt for a refresh when connectivity is next available; still serve the stale bundle rather than nothing.

### 3.5 Language Handling

- **Unsupported language requested:** Fall back to English ( `en` ) as the default, and log the unsupported request for future language prioritization — never fail the request outright.
- **Mixed-language input in community reports:** Accept as-is; no translation/normalization of free-text report content in v1.

### 3.6 General Error Handling

- All API errors return a consistent shape: `{ error: { code: string, message: string, retryable: boolean } }`.
- Any failure in a non-critical enhancement (e.g., share-link generation) must never block the critical path (viewing guidance or evacuation centers).
- All user-facing error states must have a plain-language message — no raw stack traces or technical error codes shown to end users.

## 4. Constraints

### 4.1 Tech Stack

| Layer         | Choice                                                                        | Rationale                                                                                  |
|---------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Frontend      | Next.js (React), TypeScript strict mode                                       | SSR for fast first paint on slow connections; PWA support for offline                      |
| Styling       | Tailwind CSS (core utility classes only)                                      | Small bundle size, no heavy CSS framework                                                  |
| Backend       | Node.js (Next.js API routes or a lightweight Express/Fastify service)         | Single-language stack, simpler ops for a small team                                        |
| Database      | PostgreSQL (e.g., via Supabase)                                               | Relational integrity for locations/centers/reports; Supabase gives auth + realtime cheaply |
| AI Provider   | Anthropic Claude API (Claude Sonnet-class model)                              | Used only for guidance-text generation; never for safety-critical status decisions         |
| Offline / PWA | Service Worker + IndexedDB for bundle caching                                 | Required for offline-first behavior per PRD                                                |
| Hosting       | Vercel or equivalent edge platform                                            | Fast edge delivery for low-bandwidth users                                                 |
| Maps          | Lightweight tile-based map (e.g., MapLibre GL) — avoid heavy proprietary SDKs | Keep bundle size and cost low                                                              |

### 4.2 Libraries to Avoid

- No heavy client-side ML/inference libraries — all AI generation happens server-side via the Anthropic API, never on-device in v1.
- No moment.js (use native `Date` / `Intl` or a lightweight alternative like `date-fns`) — bundle size matters on 3G.
- No unvetted crowdsourced-moderation SaaS embeds — moderation logic stays in-house and auditable given the safety context.

### 4.3 Performance Limits

- Initial page load (guidance view) must render meaningful content within **3 seconds on a simulated 3G connection**.
- Guidance generation (AI call) must complete or fall back within **5 seconds**; never leave the user on an indefinite spinner.
- Total offline bundle size capped at **2MB** to keep pre-fetch viable on limited data plans.
- App shell (HTML/CSS/JS, excluding data) must stay under **300KB gzipped**.

#### 4.4 Security & Privacy

- Collect the **minimum data necessary**: no user accounts required for core functionality in v1; location is session-scoped, not persisted server-side beyond what's needed to serve the current request, unless the user explicitly opts to save a location.
- All community reports are anonymized at intake ( `clientHash` , not device ID or IP stored in plaintext).
- All traffic over HTTPS; no exceptions.
- Rate limiting on all public write endpoints ( `/api/reports` , `/api/evac-centers/:id/status` ) to prevent abuse.
- LGU admin status-update endpoint requires authenticated, role-scoped access — never open to anonymous writes.
- No storage of precise GPS coordinates beyond the active session unless explicit, separate user consent is given.

#### 4.5 Coding Style

- TypeScript strict mode everywhere; no `any` without an explicit inline justification comment.
- Functional React components with hooks; no class components.
- All API responses validated against a schema (e.g., Zod) at the boundary — never trust client input or upstream bulletin data blindly.
- Feature-based folder structure ( `/features/guidance` , `/features/evac-centers` , `/features/reports` ), not type-based ( `/components` , `/utils` dumping grounds).
- All dates handled and stored in UTC (ISO 8601); convert to local display time only at render.

---

## 5. Acceptance Criteria

### Guidance Generation

- **Given** an active PAGASA bulletin for a user's barangay, **when** the user requests guidance, **then** the response includes AI-generated, plain-language action items in the user's selected language, attributed to that bulletin, within 5 seconds.
- **Given** no active bulletin exists for a location, **when** guidance is requested, **then** the response has `isFallback: true` and generic (non-hazard-specific) preparedness content, with no fabricated hazard claims.
- **Given** the AI generation call times out or errors, **when** guidance is requested, **then** the user receives static fallback content matching the bulletin's `hazardType` and `severity` , not an error screen.
- **Given** a bulletin's `validUntil` has passed, **when** guidance is requested, **then** that bulletin is excluded from the response and does not appear as active.

### Evacuation Centers

- **Given** an evacuation center has an LGU-admin-set status of `OPEN` , **when** 2 community reports claim `FULL` , **then** the displayed status remains `OPEN` with a supplementary note referencing the conflicting reports.
- **Given** an evacuation center has no status data at all, **when** displayed to a user, **then** its status shows as `UNKNOWN` , never `OPEN` .

- **Given** no evacuation centers exist within 5km of a user, **when** the user searches, **then** the search radius automatically expands and results (or a clear "none found within 30km" message) are returned rather than an empty result with no explanation.

### Community Reporting

- **Given** a client has already submitted 10 reports in the past hour, **when** they submit an 11th, **then** the request is rejected with a clear rate-limit error, not silently dropped.
- **Given** a report is missing a required field for its type (e.g., `EVAC_STATUS` without `targetEvacCenterId` ), **when** submitted, **then** the API returns a specific validation error identifying the missing field.

### Offline Behavior

- **Given** a user has previously loaded the app while online, **when** their connection drops, **then** the app automatically switches to offline mode and displays cached guidance with a visible "offline" indicator and last-updated timestamp.
- **Given** a user has never been online in the app, **when** they open it without connectivity, **then** they see an explicit message instructing them to connect briefly, not a blank or broken screen.

### Language

- **Given** a user selects Bisaya ( `ceb` ) as their language, **when** guidance is generated, **then** all guidance text (summary and action items) is returned in Bisaya, not mixed or defaulted to English.
- **Given** a language code not in the supported set is requested, **when** guidance is generated, **then** the response defaults to English and the event is logged, without failing the request.

### Security

- **Given** an unauthenticated request attempts to `PATCH /api/evac-centers/:id/status` , **when** it is received, **then** the API rejects it with a 401/403 and makes no data change.
- **Given** a community report is submitted, **when** it is stored, **then** no raw device identifier or IP address is persisted in plaintext alongside the report content.